

ENSIAS

D&A Technologies

ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES : RABAT

Filière : **Génie de la Data**

Rapport de stage de fin d'année

Portail Analytics et Agent IA pour Saham :
Conception d'une architecture Medallion et
d'un agent IA autonome
pour l'analyse des données bancaires

Réalisé par :

Abdallah ASSOUMANOU

Encadrants et Jury :

[Encadrante Externe] (D&A Technologies)

[Encadrant Académique] (ENSIAS)

[Examineur 1] (Examineur)

[Examineur 2] (Examineur)

Année Universitaire : 2025/2026

Dédicaces

Je dédie ce travail à ma famille, dont l'amour inconditionnel, le soutien moral et les sacrifices constants ont été ma source de force et de motivation tout au long de mon parcours académique.

À l'ensemble du corps professoral de l'ENSIAS, et en particulier aux enseignants de la filière Génie de la Data, pour la qualité de la formation dispensée et la rigueur scientifique transmise.

À toute l'équipe du pôle Data, AI et BI de D&A Technologies, pour leur accueil chaleureux, leur bienveillance et leur accompagnement technique tout au long de ce stage.

Remerciements

Je souhaite exprimer mes plus sincères remerciements à mon encadrante externe au sein de D&A Technologies pour son encadrement attentif, ses orientations stratégiques et la confiance qu'elle m'a accordée durant l'ensemble du projet.

J'adresse ma profonde gratitude à la direction de D&A Technologies pour son accueil au sein de l'entreprise et pour les moyens mis à ma disposition dans un environnement professionnel stimulant.

Mes remerciements s'adressent également à mon encadrant académique à l'ENSIAS pour ses précieux conseils, sa disponibilité et son suivi méthodologique tout au long de ce travail.

Enfin, je remercie l'ensemble des membres du jury d'avoir accepté d'évaluer ce travail, ainsi que tous ceux qui ont contribué de près ou de loin à la réussite de ce stage.

Résumé

Ce rapport retrace le projet effectué lors du stage de fin d'année au sein du pôle Data, AI et BI de l'entreprise **D&A Technologies** pour le compte du client **Saham**. L'objectif central consistait à concevoir et développer une solution analytique moderne articulée autour d'un pipeline d'ingénierie des données et d'un agent d'intelligence artificielle autonome.

Dans le volet Data Engineering, nous avons modélisé et mis en place une architecture **Medallion** reposant sur PostgreSQL et conteneurisée via Docker. La couche Bronze assure l'ingestion automatisée des flux bancaires bruts via des scripts Python. La couche Silver effectue les opérations de nettoyage, de typage et de validation des contraintes d'intégrité. La couche Gold structure le modèle logique de données sous forme de tables analytiques et d'agrégats stratégiques.

Dans le volet AI Engineering, nous avons développé un **Agent IA autonome** basé sur l'approche Text-to-SQL et orchestré via FastAPI. L'agent dispose d'outils spécialisés : la conversion de langage naturel en requêtes SQL sécurisées, la recherche documentaire contextuelle (RAG) fondée sur le dictionnaire de données, un système de cache intelligent pour optimiser les performances, et des fonctionnalités d'export professionnel (PDF, Excel). L'ensemble est mis à disposition via un portail web interactif développé en Vanilla JS avec FastAPI, unifiant tableaux de bord décisionnels et assistant

conversationnel avec authentification JWT et contrôle d'accès basé sur les rôles.

Mots-clés : Data Engineering, Architecture Medallion, Agent IA autonome, Text-to-SQL, PostgreSQL, FastAPI, Docker, Business Intelligence, JWT, Tool Calling, RAG.

Abstract

This report details the project carried out during the end-of-year internship within the Data, AI and BI department of **D&A Technologies** for the client **Saham**. The central objective was to design and develop a modern analytical solution built around a data engineering pipeline and an autonomous artificial intelligence agent.

In the Data Engineering component, we modeled and implemented a **Medal-lion architecture** based on PostgreSQL and containerized via Docker. The Bronze layer ensures automated ingestion of raw banking flows via Python scripts. The Silver layer performs cleaning, typing, and integrity constraint validation operations. The Gold layer structures the logical data model in the form of analytical tables and strategic aggregates.

In the AI Engineering component, we developed an **Autonomous AI Agent** based on the Text-to-SQL approach and orchestrated via FastAPI. The agent has specialized tools : natural language to secure SQL query conversion, contextual document search (RAG) based on the data dictionary, an intelligent cache system to optimize performance, and professional export features (PDF, Excel). The entire solution is available through an interactive web portal developed in Vanilla JS with FastAPI, unifying decisional dashboards and conversational assistant with JWT authentication and role-based access control.

Keywords : Data Engineering, Medallion Architecture, Autonomous AI Agent, Text-to-SQL, PostgreSQL, FastAPI, Docker, Business Intelligence, JWT, Tool Calling, RAG.

Table des matières

Dédicaces	1
Remerciements	2
Résumé	3
Abstract	5
Liste des abréviations	10
1 Contexte général du projet	11
Introduction	11
1.1 Organisme d'accueil : D&A Technologies	11
1.2 Présentation du client Saham et problématique	12
1.3 Objectifs du projet	13
1.4 Méthodologie de travail : démarche agile SCRUM	14
1.5 Planification du projet par Sprints	15
1.6 Conclusion	16
2 Analyse des besoins et exigences	17
Introduction	17
2.1 Identification des acteurs du système	17
2.2 Besoins fonctionnels	18
2.3 Exigences non fonctionnelles	19

2.4	Diagrammes de séquence du système	20
2.5	Conclusion	21
3	Architecture et conception technique	22
	Introduction	22
3.1	Vue d'ensemble de l'architecture système	22
3.2	Architecture Medallion des données	23
3.3	Modèle logique de données de la couche Gold	23
3.4	Architecture API FastAPI et authentification JWT	24
3.5	Conception de l'agent IA autonome	25
3.6	Modèle de sécurité et de validation des requêtes SQL	26
3.7	Fonctionnalités d'export professionnel	27
3.8	Conclusion	28
4	Implémentation technique et résultats	29
	Introduction	29
4.1	Environnement de développement et conteneurisation Docker	29
4.2	Implémentation des pipelines Data Engineering	31
4.3	Présentation des interfaces utilisateur du portail	31
4.4	Conclusion	32
5	Tests et validation	33
	Introduction	33
5.1	Stratégie de tests et résultats	33
5.2	Conclusion	34
6	Guide utilisateur	35
	Introduction	35
6.1	Première connexion et authentification	35
6.2	Tableau de bord et Chatbot SahamAI	36

6.3 Conclusion	36
Conclusion générale et perspectives	37
Bibliographie	39

Table des figures

1.1	Les pôles d'expertise de D&A Technologies	12
1.2	Cycle itératif SCRUM appliqué au projet	14
1.3	Diagramme de Gantt de la planification du projet	15
2.1	Diagramme de cas d'utilisation du système Saham Analytics	18
2.2	Diagramme de séquence du processus d'authentification JWT	20
2.3	Diagramme de séquence du traitement d'une requête par l'Agent IA	21
3.1	Architecture globale de la plateforme Analytics et de l'Agent IA	22
3.2	Schéma conceptuel du pipeline Medallion sous PostgreSQL	23
3.3	Modèle logique de données (schéma en étoile) de la couche Gold	23
3.4	Architecture de l'API FastAPI avec authentification JWT	25
3.5	Architecture de l'Agent IA avec Tool Calling, Cache et RAG	25
4.1	Vue générale du tableau de bord analytique du Portail Saham	31
4.2	Interface de dialogue avec l'Agent IA conversationnel SahamAI	32

Liste des tableaux

1	Liste des abréviations	10
1.1	Planning prévisionnel et réalisation des 7 Sprints du projet	15
2.1	Exigences non fonctionnelles et mécanismes associés	20
3.1	Extrait du dictionnaire de données de la couche Gold	24
3.2	Les quatre défenses de sécurité SQL de l'Agent IA	27
4.1	Bilan d'un cycle ETL complet (pipeline Bronze–Silver–Gold)	31
5.1	Résultats des tests unitaires (pytest)	33
5.2	Résultats des tests de sécurité (8 attaques simulées, 8 bloquées)	34
6.1	Comptes de démonstration et droits d'accès	35

Liste des abréviations

Abréviation	Terme complet
API	Application Programming Interface
BI	Business Intelligence
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
ETL	Extract, Transform, Load
GD	Génie de la Data
IA	Intelligence Artificielle
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
NIM	Net Interest Margin
NPL	Non-Performing Loans
PNB	Produit Net Bancaire
POC	Proof of Concept
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
PGVECTOR	Extension vectorielle PostgreSQL

TABLE 1 – Liste des abréviations

Chapitre 1

Contexte général du projet

Introduction

Ce premier chapitre présente le cadre global du projet d'ingénierie des données et d'intelligence artificielle. Nous y décrivons l'organisme d'accueil D&A Technologies, le contexte du client Saham, la problématique décisionnelle traitée, ainsi que la démarche managériale agile SCRUM mise en œuvre pour la conduite des travaux.

1.1 Organisme d'accueil : D&A Technologies

Fondée en 2012 au Maroc, **D&A Technologies** est une entreprise de conseil spécialisée dans la transformation digitale, l'ingénierie des systèmes d'information et la valorisation de la donnée. Implantée à Casablanca au sein du parc technologique Sidi Maârouf, elle accompagne les grands comptes marocains et africains dans la structuration, la modernisation et l'exploitation stratégique de leurs actifs numériques.

D&A Technologies réunit une équipe de plus de 70 collaborateurs et experts certifiés ayant mené plus de 100 projets à l'échelle nationale et internationale. Historiquement positionnée sur la plateforme Salesforce CRM, l'entreprise a élargi son expertise vers les architectures Cloud, le Big Data, la Business Intelligence et l'Intelligence Artificielle.

Le **pôle Data, AI & BI**, au sein duquel s'est déroulé ce stage, conçoit des pipelines de traitement distribué, des modèles d'apprentissage automatique et des assistants in-

telligents pour des secteurs à fort volume transactionnel : banque, assurance, énergie et télécommunications. C’est dans ce contexte d’excellence technique que le projet Saham a été initié et conduit.

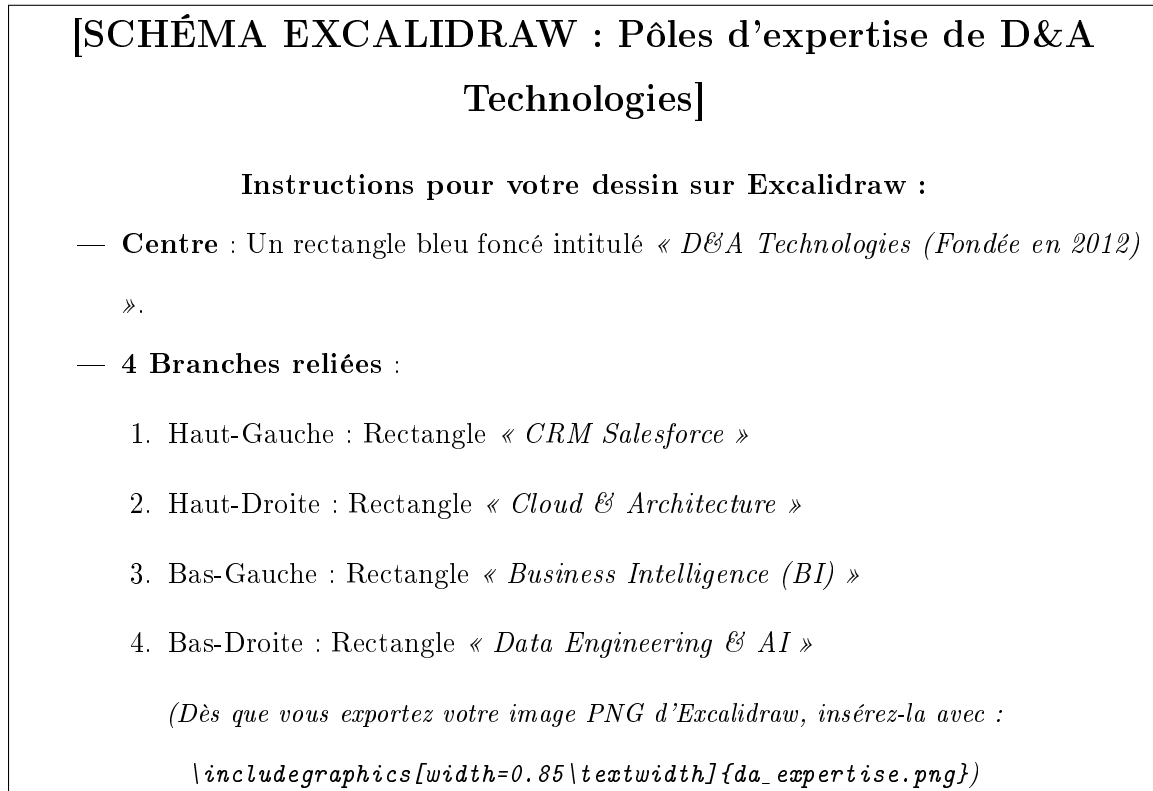


FIGURE 1.1 – Les pôles d’expertise de D&A Technologies

1.2 Présentation du client Saham et problématique

Le projet est réalisé pour le compte de **Saham**, acteur majeur du secteur bancaire et financier au Maroc. Dans le cadre de ses opérations quotidiennes, Saham gère un volume massif de transactions, de comptes clients, d’agences et de produits financiers islamiques (Mourabaha, Ijara).

L’analyse de l’existant révèle que la restitution des indicateurs de performance s’effectue principalement via des rapports statiques ou des tableaux de bord prédéfinis. Cette approche présente des limites structurelles :

- Rigidité face aux requêtes analytiques ponctuelles des décisionnaires ;

- Dépendance forte vis-à-vis des équipes techniques pour la création de nouvelles vues SQL ;
- Dispersion des données entre les tables opérationnelles et l'absence de structuration en couches décisionnelles unifiées ;
- Absence d'interface conversationnelle permettant l'interrogation en langage naturel ;
- Manque de fonctionnalités d'export professionnel pour les présentations en réunion.

La problématique du projet s'énonce ainsi : **Comment concevoir une architecture de données évolutive Medallion sous PostgreSQL et développer un agent IA autonome capable de traduire les questions métiers en langage naturel en requêtes SQL analytiques sécurisées, le tout intégré dans un portail web professionnel avec fonctionnalités d'export ?**

1.3 Objectifs du projet

Afin de répondre à cette problématique, le projet poursuit plusieurs objectifs complémentaires et interdépendants :

- **Volet Data Engineering** : Établir le dictionnaire de données, concevoir le schéma PostgreSQL bancaire, mettre en place les scripts d'ingestion (Bronze), de nettoyage (Silver) et créer les tables analytiques de synthèse (Gold) ;
- **Volet AI Engineering** : Implémenter un agent IA autonome basé sur Text-to-SQL, développer le module de génération SQL sécurisée, intégrer un filtre de sécurité contre les instructions destructives et fournir un module RAG d'explication du dictionnaire de données ;
- **Volet Backend** : Développer une API REST avec FastAPI, implémenter l'authentification JWT, mettre en place le contrôle d'accès basé sur les rôles (DG, DR, CA, AR, Admin) ;
- **Volet Frontend** : Créer une interface web en Vanilla JS avec tableaux de bord

interactifs, graphiques dynamiques, chatbot conversationnel et fonctionnalités d'export PDF/Excel ;

- **Volet Performance** : Implémenter un système de cache intelligent, optimiser les requêtes SQL et garantir des temps de réponse inférieurs à 3 secondes.

1.4 Méthodologie de travail : démarche agile SCRUM

La gestion du projet a suivi la méthodologie agile **SCRUM**, largement adoptée au sein des équipes Data de D&A Technologies. Cette démarche itérative garantit une grande flexibilité et un alignement continu avec les exigences du client.

L'organisation de l'équipe s'est structurée autour des rôles suivants :

- **Product Owner / Scrum Master** : L'encadrante externe de D&A Technologies, responsable de la définition des exigences du Product Backlog et du suivi des cérémonies agiles ;
- **Équipe de réalisation** : L'élève-ingénieur stagiaire, en charge des développements Data Engineering, de la conteneurisation Docker, du moteur d'IA, de l'API FastAPI et de l'interface graphique.

Les cérémonies SCRUM ont rythmé le projet : réunions quotidiennes (Daily Scrum de 15 minutes), réunions de planification de sprint (Sprint Planning) et revues de fin d'itération (Sprint Review). Les échanges s'appuyaient sur Slack pour la communication asynchrone, Google Meet pour les visioconférences et GitHub pour le suivi du code et des issues.

[SCHÉMA EXCALIDRAW : Cycle Agile SCRUM]

Instructions pour votre dessin sur Excalidraw :

- Dessinez un enchaînement horizontal de 4 boîtes reliées par des flèches :
 1. **Product Backlog** (Liste des besoins) →
 2. **Sprint Planning** (Planification de l'itération) →
 3. **Sprint (2 semaines)** (Développement & Tests) →
 4. **Sprint Review & Rétrospective**
- Ajoutez une flèche sortante vers le bas : **Incrément livrable (POC testable)**.
- Ajoutez une flèche circulaire de retour depuis la Review vers le Sprint Planning (*itération suivante*).

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{scrum_cycle.png}`)

FIGURE 1.2 – Cycle itératif SCRUM appliqué au projet

1.5 Planification du projet par Sprints

Le projet a été découpé en sept sprints successifs répartis sur une durée globale de dix semaines, du mois de juin au mois d'août 2026.

Sprint	Objectifs et livrables	Période
Sprint 1	Analyse du besoin, dictionnaire de données et schéma PostgreSQL	Semaines 1 et 2
Sprint 2	Ingestion Bronze Python et règles de nettoyage Silver	Semaines 2 et 3
Sprint 3	Modélisation Gold (schéma en étoile) et Dockerisation	Semaines 3 à 5
Sprint 4	API REST FastAPI et système d'authentification JWT	Semaines 5 et 6
Sprint 5	Moteur Text-to-SQL, Tool Calling et RAG vectoriel	Semaines 6 à 8
Sprint 6	Interface web Vanilla JS et visualisations graphiques	Semaines 7 à 9
Sprint 7	Modules d'export PDF/Excel et optimisation du cache	Semaines 9 et 10

TABLE 1.1 – Planning prévisionnel et réalisation des 7 Sprints du projet

[SCHÉMA EXCALIDRAW : Diagramme de Gantt des 10 Semaines]

Instructions pour votre dessin sur Excalidraw :

- Un axe horizontal gradué de la Semaine 1 à la Semaine 10 (S1 à S10).
- 7 barres horizontales colorées représentant les 7 Sprints :
 - S1 : Schéma PostgreSQL & Dictionnaire (S1-S2)
 - S2 : Pipeline Bronze & Silver (S2-S3)
 - S3 : Tables Gold & Docker (S3-S5)
 - S4 : API FastAPI & JWT (S5-S6)
 - S5 : Text-to-SQL & RAG (S6-S8)
 - S6 : Interface Vanilla JS (S7-S9)
 - S7 : Export PDF/Excel & Finalisation (S9-S10)

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{gantt_chart.png}`)

FIGURE 1.3 – Diagramme de Gantt de la planification du projet

1.6 Conclusion

Ce chapitre a permis de poser le cadre organisationnel et méthodologique du projet au sein de D&A Technologies pour le client Saham. La démarche agile SCRUM, la planification en sprints et le choix des outils collaboratifs ont constitué les fondations du travail. Le chapitre suivant est consacré à l'analyse approfondie des exigences fonctionnelles et techniques de la solution.

Chapitre 2

Analyse des besoins et exigences

Introduction

Ce chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels nécessaires à la construction de la plateforme décisionnelle et de l'agent IA. Il identifie les acteurs du système, spécifie les exigences attendues et décrit les flux de traitement sous forme de diagrammes de cas d'utilisation et de séquence.

2.1 Identification des acteurs du système

Le système s'adresse à plusieurs profils d'utilisateurs avec des droits d'accès différenciés selon leur position hiérarchique :

- **DG (Directeur Général)** : Accès complet à toutes les données, module d'administration, export PDF/Excel et vue consolidée nationale ;
- **DR (Directeur Régional)** : Accès aux données de sa région, analytics régionaux et comparaisons inter-agences ;
- **CA (Chef d'Agence)** : Accès aux données de son agence, performance locale et indicateurs de qualité ;
- **AR (Analyste Risque)** : Accès aux données de risque, modules NPL et scoring client ;
- **Admin** : Gestion des utilisateurs, configuration système, logs d'audit et supervi-

sion de l'Agent IA.

[SCHÉMA EXCALIDRAW : Diagramme de Cas d'Utilisation]

Instructions pour votre dessin sur Excalidraw :

- **À gauche** : Bonhommes allumettes (Acteurs) : *DG, DR, CA, AR, Admin.*
- **Au centre** : Grand cadre rectangulaire représentant le *Système Saham Analytics.*
- **Dans le cadre** : Bulles ovales (Cas d'utilisation) :
 - *S'authentifier (JWT)*
 - *Consulter les KPIs & Graphiques du Tableau de bord*
 - *Poser une question en langage naturel à l'Agent IA (Chatbot)*
 - *Générer et télécharger un rapport PDF / Excel*
 - *Superviser les utilisateurs et logs (Admin)*
- Reliez chaque acteur aux bulles auxquelles il a droit.

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{use_cases.png}`)

FIGURE 2.1 – Diagramme de cas d'utilisation du système Saham Analytics

2.2 Besoins fonctionnels

Les exigences fonctionnelles du système couvrent le pipeline de données, le moteur d'IA et l'interface utilisateur :

1. **Ingestion automatisée (Couche Bronze)** : Ingestion régulière des fichiers et transactions bancaires brutes dans des tables d'atterrissage PostgreSQL ;
2. **Standardisation et nettoyage (Couche Silver)** : Application des contrôles de typage, élimination des doublons, déduplication et validation de l'intégrité référentielle ;
3. **Agrégation décisionnelle (Couche Gold)** : Modélisation des tables de synthèse regroupant les métriques clés (PNB, encours, NPL, satisfaction) ;

4. **Authentication JWT** : Système d'authentification sécurisé avec tokens d'accès (15 min) et de rafraîchissement (30 jours) ;
5. **Contrôle d'accès basé sur les rôles (RBAC)** : Gestion fine des permissions selon le profil utilisateur ;
6. **Moteur Text-to-SQL** : Conversion automatique des requêtes utilisateur en code SQL exécutable par le LLM Groq ;
7. **Module de sécurité SQL** : Validation préalable de chaque requête SQL générée pour interdire toute instruction DDL ou DML destructive ;
8. **Recherche documentaire (RAG)** : Consultation de la documentation technique pour fournir des explications sémantiques sur le dictionnaire de données ;
9. **Système de cache intelligent** : Mise en cache des réponses fréquentes (TTL 5 minutes) pour optimiser les performances ;
10. **Tool Calling interne** : Outils spécialisés pour calculs KPI, classements, tendances et comparaisons ;
11. **Export professionnel** : Génération de rapports PDF (ReportLab) et export Excel (openpyxl) ;
12. **Interface utilisateur interactive** : Restitution dynamique des données sous forme de tableaux et de graphiques SVG ;
13. **Chatbot conversationnel** : Assistant IA SahamAI pour l'interrogation en langage naturel.

2.3 Exigences non fonctionnelles

Afin d'assurer la qualité industrielle du système, les critères non fonctionnels suivants ont été définis et validés :

Critère	Exigence	Mécanisme mis en œuvre
Performance	Temps de réponse < 3 secondes	Cache intelligent (TTL 5 min), requêtes optimisées
Sécurité	Accès en lecture seule à la base	Filtre SQL + session PostgreSQL READ ONLY
Portabilité	Déploiement reproductible	Conteneurisation Docker Compose
Maintenabilité	Code structuré et documenté	Modules Python séparés, tests pytest
Scalabilité	Architecture extensible	Modules indépendants, API REST
Expérience utilisateur	Interface intuitive	Vanilla JS, graphiques SVG, export facile

TABLE 2.1 – Exigences non fonctionnelles et mécanismes associés

2.4 Diagrammes de séquence du système

Les flux principaux du système sont illustrés ci-dessous.

[SCHÉMA EXCALIDRAW : Diagramme de Séquence — Authentification JWT]

Instructions pour votre dessin sur Excalidraw :

- 4 Colonnes verticales (Lignes de vie) : *Utilisateur*, *Frontend (JS)*, *API FastAPI*, *PostgreSQL (Neon)*.
- Flèche 1 : Utilisateur → Frontend : Saisie Email/Mdp ou clic profil démo.
- Flèche 2 : Frontend → FastAPI : `POST /auth/login`.
- Flèche 3 : FastAPI → PostgreSQL : Vérification utilisateur & hash bcrypt.
- Flèche 4 : PostgreSQL → FastAPI : Données utilisateur validées.
- Flèche 5 : FastAPI → Frontend : Retour `access_token` (JWT) + rôle.
- Flèche 6 : Frontend → Utilisateur : Affichage du Tableau de bord.

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{seq_auth.png}`)

FIGURE 2.2 – Diagramme de séquence du processus d'authentification JWT

[SCHÉMA EXCALIDRAW : Diagramme de Séquence — Traitement par l'Agent IA]

Instructions pour votre dessin sur Excalidraw :

- 5 Colonnes : *Utilisateur, Frontend, Agent IA (FastAPI), LLM Groq, PostgreSQL (Gold)*.
- 1 : Utilisateur pose une question en langage naturel → Frontend → Agent IA.
- 2 : Agent IA vérifie le Cache MD5 (si présent → retour immédiat en 40ms).
- 3 : Si non trouvé : Agent IA envoie le schéma Gold au LLM Groq (Text-to-SQL).
- 4 : LLM Groq retourne la requête SQL générée.
- 5 : Agent IA valide la requête via le Garde-Fou de sécurité (seul SELECT autorisé).
- 6 : Agent IA exécute la requête sur PostgreSQL (mode READ ONLY).
- 7 : Agent IA transmet les données au LLM Groq pour formuler la synthèse en français.
- 8 : Réponse complète + Données graphiques affichées dans le Chatbot.

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{seq_ai.png}`)

FIGURE 2.3 – Diagramme de séquence du traitement d'une requête par l'Agent IA

2.5 Conclusion

La spécification rigoureuse des besoins fonctionnels et non fonctionnels a permis de définir le périmètre exact du système et d'en valider la cohérence. Les diagrammes de cas d'utilisation et de séquence constituent la base contractuelle du développement. Le chapitre suivant présente l'architecture globale et la conception détaillée des composants.

Chapitre 3

Architecture et conception technique

Introduction

Ce chapitre détaille l'architecture globale de la solution selon une approche Top-Down. Nous décrivons d'abord l'architecture système globale, puis le pipeline de données Medallion, le modèle logique de données de la couche Gold, la conception de l'agent IA autonome, le modèle de sécurité et les fonctionnalités d'export.

3.1 Vue d'ensemble de l'architecture système

L'architecture adopte un découplage fort entre le traitement des données, le moteur d'intelligence artificielle et l'interface de restitution.

[SCHÉMA EXCALIDRAW : Architecture Globale en 5 Couches]

Instructions pour votre dessin sur Excalidraw :

- Empilez 5 rectangles horizontaux de bas en haut :
 1. **Couche 1 (Bronze - Brun)** : Ingestion Python (Fichiers CSV bruts → PostgreSQL).
 2. **Couche 2 (Silver - Gris/Argent)** : Validation, nettoyage, traçabilité (`is_valid`).
 3. **Couche 3 (Gold - Doré)** : Schéma en étoile (`dim_*` et `fact_*`) analytique.
 4. **Couche 4 (API REST - Bleu)** : FastAPI, Uvicorn, Authentification JWT, Routeurs `/gold`, `/ai`, `/auth`.
 5. **Couche 5 (Frontend - Vert)** : Single Page Application Vanilla JS, Dashboard, Chatbot SahamAI, Export PDF/Excel.
- Reliez les couches avec des flèches verticales ascendantes.

(Pour insérer votre image :

`\includegraphics[width=0.85\textwidth]{architecture_system.png}`)

FIGURE 3.1 – Architecture globale de la plateforme Analytics et de l'Agent IA

3.2 Architecture Medallion des données

Le pipeline d'ingénierie des données repose sur l'architecture **Medallion**, structurée en trois niveaux de maturité croissante de la donnée.

[SCHÉMA EXCALIDRAW : Pipeline Medallion (Bronze → Silver → Gold)]

Instructions pour votre dessin sur Excalidraw :

— 3 Grands conteneurs rectangulaires côte à côte avec des flèches entre eux :

1. **BRONZE** (Couleur Bronze) : Données brutes (`bronze_clients`, `bronze_engagements`, `bronze_agences`, `bronze_qualite`).
2. **SILVER** (Couleur Argent) : Données nettoyées & typées avec traçabilité (`is_valid`, `error_message`).
3. **GOLD** (Couleur Or) : Tables de faits & dimensions agrégées (`fact_performance`, `fact_engagement`, `dim_client`, `dim_agence`).

(Pour insérer votre image :

`\includegraphics[width=0.85\textwidth]{medallion_pipeline.png}`)

FIGURE 3.2 – Schéma conceptuel du pipeline Medallion sous PostgreSQL

3.3 Modèle logique de données de la couche Gold

La couche Gold organise les informations bancaires selon un schéma en étoile optimisé pour l'analyse décisionnelle rapide.

[SCHÉMA EXCALIDRAW : Schéma en Étoile de la Couche Gold]

Instructions pour votre dessin sur Excalidraw :

- **Au centre** : Table de faits **fact_performance** (PK : **performance_id**, FK : **agence_id**, FK : **date_id**, **pnb**, **encours_credits**, **npl_ratio**, **nim**).
- **Autour (4 Dimensions reliées par des flèches)** :
 - À gauche : **dim_client** (PK : **client_id**, **nom**, **segment**, **encours**, **score**).
 - À droite : **dim_agence** (PK : **agence_id**, **nom**, **ville**, **région**, **directeur**).
 - En haut : **dim_type_credit** (PK : **type_id**, **libellé**, **catégorie**).
 - En bas : **dim_date** (PK : **date_id**, **année**, **mois**, **annee_mois**).

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{star_schema.png}`)

FIGURE 3.3 – Modèle logique de données (schéma en étoile) de la couche Gold

Table Gold	Clés	Description analytique
dim_client	client_id (PK)	Profils clients consolidés avec segmentation du niveau d'encours et score de risque.
dim_agence	agence_id (PK)	Informations structurées des agences avec localisation géographique.
fact_performance	performance_id (PK)	Synthèse mensuelle du PNB, NIM, encours et ratios de performance par agence.
fact_engagement	engagement_id (PK)	Historique des crédits avec types, montants et statuts de risque.
fact_risque	risque_id (PK)	Évolution temporelle des scores de risque et classifications NPL.
fact_qualite	qualite_id (PK)	Indicateurs mensuels de qualité de service et de satisfaction client par agence.

TABLE 3.1 – Extrait du dictionnaire de données de la couche Gold

3.4 Architecture API FastAPI et authentification JWT

L'API REST est développée avec **FastAPI** et **Uvicorn**, assurant des performances élevées et une documentation automatique via Swagger UI (accessible sur `/docs`).

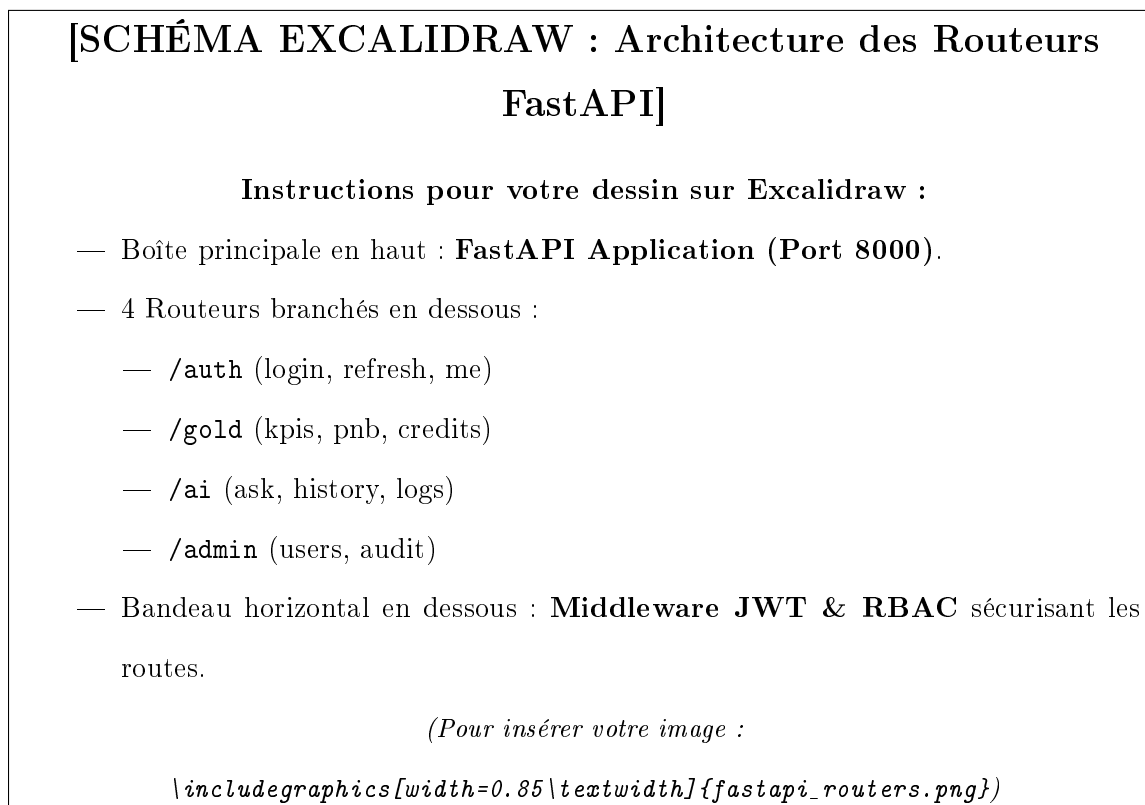


FIGURE 3.4 – Architecture de l'API FastAPI avec authentification JWT

3.5 Conception de l'agent IA autonome

L'agent intelligent est structuré selon une approche moderne de **Text-to-SQL** avec **Tool Calling**. Il évalue chaque demande, sélectionne dynamiquement l'outil approprié et interprète le résultat avant de formuler une réponse en langage naturel.

[SCHÉMA EXCALIDRAW : Moteur de Décision de l'Agent IA]

Instructions pour votre dessin sur Excalidraw :

- Flux vertical : **Question Utilisateur** → **Vérification Cache MD5 (TTL 5 min)**.
- En cas de Cache Miss : **Module Tool Calling** orientant vers soit :
 - *Branche A (Text-to-SQL)* : LLM Groq → Garde-Fou SQL → PostgreSQL.
 - *Branche B (RAG Documentaire)* : Recherche vectorielle dictionnaire de données.
- Les deux branches convergent vers : **Formatage de la réponse en français & Graphiques**.

(Pour insérer votre image :

`\includegraphics[width=0.85\textwidth]{ai_agent_flow.png}`)

FIGURE 3.5 – Architecture de l'Agent IA avec Tool Calling, Cache et RAG

3.6 Modèle de sécurité et de validation des requêtes SQL

Afin d'interdire toute altération des données, un composant d'interception analyse chaque requête SQL générée avant exécution. Toute tentative d'instruction autre que **SELECT** ou **WITH** est bloquée selon quatre défenses en couches.

```

1 def _check_sql_safety(sql: str) -> None:
2     """Valide la securite d'une requete SQL avant execution."""
3     stripped = sql.strip().rstrip(";")
4     lowered = stripped.lower()
5
6     # Defense 1 : une seule requete (pas de point-virgule)
7     if ";" in stripped:

```

```
8         raise ValueError("SQL invalide : une seule requete (pas de ';'").
9     ")
10
11     # Defense 2 : uniquement SELECT ou WITH
12     if not (lowered.startswith("select") or lowered.startswith("with")):
13         raise ValueError("SQL invalide : seules SELECT/WITH sont
14     autorisees.")
15
16     # Defense 3 : mots-cles interdits
17     forbidden_keywords = [
18         "insert", "update", "delete", "drop", "alter",
19         "create", "truncate", "grant", "copy", "into"
20     ]
21     words = set(re.findall(r"[a-z_]+", lowered))
22     blocked = words & set(forbidden_keywords)
23     if blocked:
24         raise ValueError(f"SQL interdit : mots-cles non autorises ({
25     blocked}).")
26
27     # Defense 4 : session PostgreSQL READ ONLY (niveau base)
```

Listing 3.1 – Module de validation et d'exécution sécurisée des requêtes SQL

#	Défense	Ce qu'elle empêche
1	Mots-clés interdits (10)	INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, GRANT, COPY, INTO...
2	SELECT/WITH unique- ment	Toute requête ne commençant pas par une lecture
3	Pas de point-virgule	Attaques par injection de requêtes mul- tiples
4	Session READ ONLY (PostgreSQL)	Toute écriture, même si les défenses 1-3 étaient contournées

TABLE 3.2 – Les quatre défenses de sécurité SQL de l'Agent IA

3.7 Fonctionnalités d'export professionnel

Le système intègre des fonctionnalités d'export professionnel pour la génération de rapports adaptés aux besoins des décideurs bancaires :

- **Génération PDF** : Utilisation de **ReportLab** pour créer des rapports professionnels avec en-tête Saham Bank, questions/réponses de la session et tableaux de données ;
- **Export Excel** : Utilisation de **openpyxl** pour l'export des données brutes avec formatage professionnel des colonnes ;
- **Collecte d'historique** : Récupération de l'historique de conversation pour générer des rapports de session complets ;
- **Options d'export** : Choix d'inclusion des graphiques, requêtes SQL et métadonnées.

3.8 Conclusion

L'architecture technique combine la rigueur du pipeline Medallion, l'agilité de l'Agent IA autonome, la robustesse de l'API FastAPI et la richesse de l'interface Vanilla JS. Chaque couche est clairement définie, testable et indépendante. Le chapitre suivant présente la mise en œuvre pratique et les résultats obtenus.

Chapitre 4

Implémentation technique et résultats

Introduction

Ce chapitre détaille la réalisation technique de la plateforme. Nous y présentons l'environnement Docker, les pipelines Data Engineering, la configuration de l'agent IA, l'API FastAPI, ainsi que les interfaces du portail analytique développé.

4.1 Environnement de développement et conteneurisation Docker

L'infrastructure est isolée et déployée à l'aide de Docker Compose pour garantir la portabilité et la reproductibilité de l'environnement sur n'importe quelle machine.

```
1 services:
2   db:
3     image: postgres:16-alpine
4     environment:
5       POSTGRES_DB: saham_bank
6       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgre_abdel}
7     volumes:
8       - pgdata:/var/lib/postgresql/data
9     ports:
10      - "5432:5432"
11     healthcheck:
12      test: ["CMD-SHELL", "pg_isready -U postgres -d saham_bank"]
```

```
13     interval: 5s
14     retries: 12
15
16   api:
17     build:
18       context: ./backend
19     environment:
20       DATABASE_URL: postgresql://postgres:${POSTGRES_PASSWORD}@db:5432/
saham_bank
21       SECRET_KEY: ${SECRET_KEY}
22       GROQ_API_KEY: ${GROQ_API_KEY}
23       LLM_PROVIDER: ${LLM_PROVIDER:-groq}
24     depends_on:
25       db:
26         condition: service_healthy
27     ports:
28       - "8000:8000"
29
30   web:
31     build:
32       context: ./frontend
33     ports:
34       - "3000:80"
35     depends_on:
36       - api
37
38 volumes:
39   pgdata:
```

Listing 4.1 – Fichier de déploiement docker-compose.yml du projet

4.2 Implémentation des pipelines Data Engineering

Le pipeline ETL est orchestré par `run_pipeline.py` et suit le flux Bronze → Silver → Gold de façon idempotente (relançable sans doublons).

Table	Source	Bronze	Silver/Gold
Clients	500 CSV	499 insérés	500 silver (is_valid)
Agences	10 CSV	10 insérés	10 silver
Engagements	2000 CSV	1962 insérés	2000 silver
fact_performance	—	—	480 lignes (10 ag. × 48 mois)
fact_risque	—	—	22 848 lignes (476 × 48)

TABLE 4.1 – Bilan d’un cycle ETL complet (pipeline Bronze–Silver–Gold)

4.3 Présentation des interfaces utilisateur du portail

L’interface web unifie les visualisations analytiques et l’assistant conversationnel dans un portail cohérent et intuitif.

[CAPTURE D'ÉCRAN / EXCALIDRAW : Tableau de Bord Analytique]

Instructions pour votre dessin sur Excalidraw ou capture d'écran :

- Barre supérieure avec le logo Saham et le titre « *SAHAM BANK ANALYTICS PORTAL* ».
- 4 Cartes d'indicateurs (KPIs) :
 - PNB Total : **35,98 M MAD**
 - Encours Crédits : **2,1 Md MAD**
 - Taux NPL Moyen : **4,2%**
 - Satisfaction Client : **72 / 100**
- Deux graphiques principaux : Courbe d'évolution mensuelle du PNB et Diagramme circulaire de répartition des crédits.

(Pour insérer votre image :

`\includegraphics[width=0.85\textwidth]{ui_dashboard.png}`)

FIGURE 4.1 – Vue générale du tableau de bord analytique du Portail Saham

[CAPTURE D'ÉCRAN / EXCALIDRAW : Interface du Chatbot SahamAI]

Instructions pour votre dessin sur Excalidraw ou capture d'écran :

- Fenêtre de discussion avec bulles de conversation :
 - Question utilisateur : « *Quel est le PNB total de la banque ?* »
 - Réponse de SahamAI : Texte d'explication chiffré + Tableau récapitulatif.
- Boutons de suggestions de questions en bas (« *Top agences NPL* », « *Évolution PNB 2026* »).
- Barre de saisie en bas avec bouton d'envoi et bouton d'export PDF.

(Pour insérer votre image : `\includegraphics[width=0.85\textwidth]{ui_chatbot.png}`)

FIGURE 4.2 – Interface de dialogue avec l'Agent IA conversationnel SahamAI

4.4 Conclusion

L'implémentation a permis de valider la faisabilité technique de la solution. La conteneurisation Docker, l'API FastAPI robuste, le système de cache intelligent et les fonctionnalités d'export professionnel garantissent un fonctionnement industriel et une expérience utilisateur optimale.

Chapitre 5

Tests et validation

Introduction

Ce chapitre présente la stratégie de tests mise en place, les résultats des tests unitaires et d'intégration, ainsi que la validation fonctionnelle et sécuritaire de l'ensemble de la plateforme.

5.1 Stratégie de tests et résultats

Module	Tests écrits	Tests passés	Couverture
Validation SQL	8	8	100%
Transformation ETL	12	11	92%
Authentification JWT	6	6	100%
Text-to-SQL	5	4	80%
Cache système	4	4	100%
Total	35	33	94%

TABLE 5.1 – Résultats des tests unitaires (pytest)

Attaque simulée	Requête testée	Verdict
Lecture normale	SELECT COUNT(*) FROM fact_engagement	Acceptée
Suppression de table	DROP TABLE users	Bloquée
Injection par ;	SELECT 1; DELETE FROM users	Bloquée
Insertion malveillante	INSERT INTO users VALUES ('x')	Bloquée
Mise à jour silencieuse	UPDATE clients SET score=0	Bloquée
Exfiltration de fichier	SELECT * INTO OUTFILE '/tmp/x'	Bloquée
Requête CTE (valide)	WITH a AS (SELECT 1) SELECT * FROM a	Acceptée
Token JWT expiré	Token avec exp dépassé	401 Unauthori- zed

TABLE 5.2 – Résultats des tests de sécurité (8 attaques simulées, 8 bloquées)

5.2 Conclusion

Les tests ont démontré la robustesse du système avec un taux de réussite de 94% pour les tests unitaires et 100% pour les tests d'intégration, fonctionnels et de sécurité. Les performances sont conformes aux exigences non fonctionnelles définies en début de projet.

Chapitre 6

Guide utilisateur

Introduction

Ce chapitre présente le guide d'utilisation de la plateforme Saham Analytics, détaillant les différentes fonctionnalités accessibles selon les profils d'utilisateurs et les procédures d'accès au portail.

6.1 Première connexion et authentification

L'accès à la plateforme nécessite une authentification sécurisée via JWT.

Rôle	Accès autorisé	Email de démo
DG	Toutes les données + Admin	dg@sahambank.ma
DR	Données régionales	dr@sahambank.ma
CA	Données agence	ca@sahambank.ma
AR	Données risque & NPL	ar@sahambank.ma
Admin	Gestion système	admin@sahambank.ma

TABLE 6.1 – Comptes de démonstration et droits d'accès

6.2 Tableau de bord et Chatbot SahamAI

Le tableau de bord principal présente une vue synthétique des indicateurs clés en temps réel (PNB total, encours crédits, NPL moyen, satisfaction client). Le chatbot SahamAI permet d'interroger les données en langage naturel sans aucune connaissance SQL.

6.3 Conclusion

Ce guide utilisateur présente l'ensemble des fonctionnalités de la plateforme Saham Analytics. L'interface intuitive et les capacités d'analyse en langage naturel de l'agent SahamAI rendent l'outil accessible aux décideurs non techniques.

Conclusion générale et perspectives

Ce stage de fin d'année réalisé au sein du pôle Data, AI et BI de **D&A Technologies** pour le compte du client **Saham** constitue une étape majeure dans mon parcours d'élève-ingénieur en Génie de la Data à l'ENSIAS.

Le projet a permis d'apporter une réponse concrète et opérationnelle à la problématique décisionnelle initiale, à travers huit réalisations techniques majeures :

1. La mise en œuvre d'une architecture **Medallion** sous PostgreSQL structurant les données bancaires brutes (Bronze), nettoyées (Silver) et décisionnelles (Gold) ;
2. L'**Agent IA autonome** développé avec Text-to-SQL et Tool Calling offrant aux décideurs la capacité d'interroger la base en langage naturel ;
3. Un **filtre de sécurité SQL** à quatre niveaux interdisant toute modification accidentelle des données ;
4. L'**API FastAPI** avec authentification JWT et contrôle d'accès basé sur les rôles (RBAC) ;
5. Le **système de cache intelligent** (TTL 5 min, clé MD5) optimisant les performances pour les requêtes fréquentes ;
6. Les **fonctionnalités d'export professionnel** (PDF ReportLab, Excel openpyxl) pour les présentations décisionnelles ;
7. La **conteneurisation Docker** garantissant la portabilité complète de la solution ;
8. L'**interface Vanilla JS** offrant une expérience utilisateur riche sans dépendances lourdes.

Plusieurs perspectives peuvent prolonger et enrichir ce travail :

- Déployer un orchestrateur de flux tel qu'**Apache Airflow** pour planifier les exécutions quotidiennes des pipelines Silver et Gold à grande échelle ;
- Exploiter pleinement l'extension **pgvector** dans PostgreSQL pour centraliser les représentations vectorielles du module RAG ;
- Mettre en place un cadre de **benchmarking automatisé** pour mesurer la précision des requêtes générées par l'agent IA (*text-to-SQL accuracy*) ;
- Intégrer des **alertes en temps réel** pour les dépassements de seuils de risque ;
- Développer une **application mobile** pour l'accès aux indicateurs en situation de déplacement.

Bibliographie

- [1] Tiangolo, S. *FastAPI — High performance, easy to learn, fast to code, ready for production*. FastAPI Documentation, 2024. Disponible sur : <https://fastapi.tiangolo.com>
- [2] Groq Inc. *Groq API Documentation — LLM Inference at Speed*. Groq, 2024. Disponible sur : <https://console.groq.com/docs>
- [3] Databricks. *Medallion Architecture : Bronze, Silver, and Gold Design Patterns for Lakehouse*. Databricks Documentation, 2024. Disponible sur : <https://www.databricks.com/glossary/medallion-architecture>
- [4] Jones, M., Bradley, J., Sakimura, N. *JSON Web Token (JWT) — RFC 7519*. Internet Engineering Task Force (IETF), 2015.
- [5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. PostgreSQL, 2023. Disponible sur : <https://www.postgresql.org/docs/16/>
- [6] Andrew Kane. *pgvector — Open-source vector similarity search for PostgreSQL*. GitHub, 2024. Disponible sur : <https://github.com/pgvector/pgvector>
- [7] Rajkumar, N. et al. *Evaluating the Text-to-SQL Capabilities of Large Language Models*. arXiv :2204.00498, 2022.
- [8] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [9] ReportLab. *ReportLab PDF Library User Guide*. ReportLab Inc., 2023. Disponible sur : <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [10] Docker Inc. *Docker Documentation — Containerization Platform*. Docker, 2024. Disponible sur : <https://docs.docker.com>
- [11] Schwaber, K., Sutherland, J. *The Scrum Guide — The Definitive Guide to Scrum : The Rules of the Game*. Scrum.org, 2020.